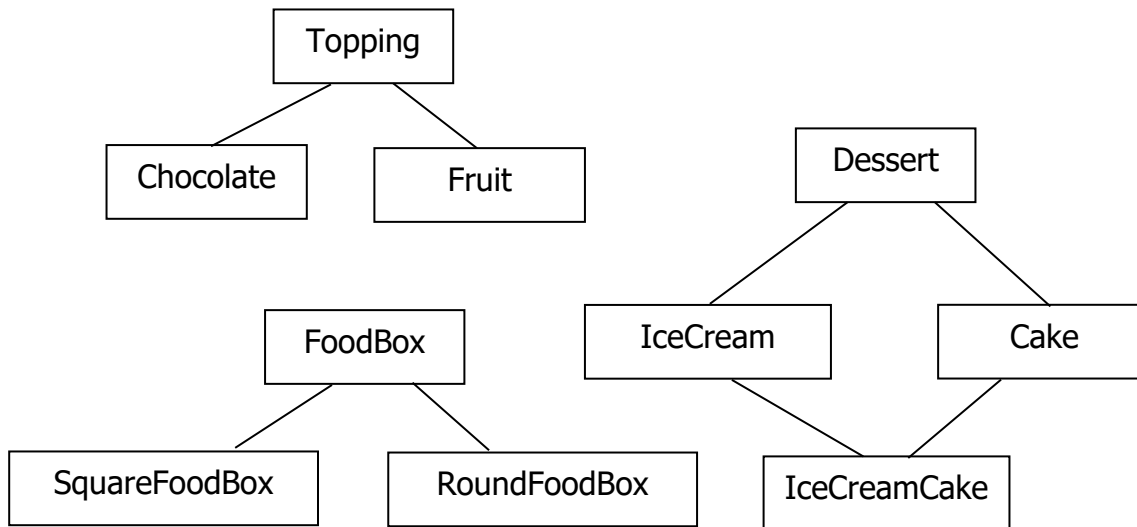


แบบฝึกหัดที่ 60

จงสร้างคลาสต่างๆ ตามตาราง โดยมีการรับทอดตามรูปและไม่เกิดความกำกวม การรับทอดทั้งหมดให้ทำให้เป็นการรับทอดแบบ public และกำหนดให้ Topping, Dessert, FoodBox เป็น abstract class โดยให้สร้างฟังก์ชัน show เป็น pure virtual function แต่ละคลาสให้เก็บข้อมูลทั้งหมดไว้ในส่วนของ private



คลาส	แอททริบิวต์	ความหมาย
Topping	string detail; double price;	รายละเอียดของท็อปปิ้ง ราคา
Chocolate	int type;	ชนิดของช็อคโกแลต เช่น 1.dark 2.white 3.milk
Fruit	string name;	ชื่อผลไม้
Dessert	int price; string flavor; Topping **topping; int num;	ราคา (ไม่รวมราคาท็อปปิ้ง) รสชาติ อาร์เรย์เก็บพอยเตอร์ที่ชี้ไปยัง Topping ขนาดของอาร์เรย์ topping
IceCream	int type;	เช่น 1.ไอศกรีมธรรมดา 2.เจลาโต้ 3.ไอศกรีมโยเกิร์ต
Cake	int type;	เช่น 1.butter 2.foam 3.mousse 4.sponge
IceCreamCake	int layer;	จำนวนเลเยอร์ (เค้ก+ไอติม)
FoodBox	Dessert** dessert; int num; string *foodDetail;	อาร์เรย์เก็บพอยเตอร์ที่ชี้ไปยัง Dessert ขนาดของอาร์เรย์ dessert และ ขนาดของอาร์เรย์ foodDetail อาร์เรย์เก็บคำอธิบายขนมหวานแต่ละชนิด
SquareFoodBox	int width; int height; int length;	ความกว้าง ความสูง ความยาว
RoundFoodBox	int diameter; int height;	ความยาวเส้นผ่าศูนย์กลาง ความสูงของกล่อง

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้
(ในกรณีที่ต้องจองพื้นที่ ให้เนศ.จองพื้นที่และคืนพื้นที่ให้เรียบร้อยด้วย ทุกๆ object จะต้องถูกสร้างและถูกทำลายได้อย่างถูกต้อง และเนศ.สามารถสร้าง attributes และ function เพิ่มเติมได้ตามความจำเป็น)

คลาส Topping, Chocolate, Fruit จงสร้าง (ไม่ให้สร้างฟังก์ชัน set)

- default constructor ในการกำหนดค่าดังนี้
 - o Topping มีค่า detail="no detail" price=0
 - o Chocolate เป็น dark , มีค่า detail="made in Swiss" , price=40
 - o Fruit เป็น สตรอเบอรี่ , มีค่า detail="sweet" , price=30
- constructor กำหนดค่าแอททริบิวต์ทุกตัวรวมทั้ง base class โดยที่ price ต้องมากกว่า 0 และ type มีแค่ 3 แบบ
- ฟังก์ชัน get ในการรีเทิร์นค่าของแอททริบิวต์แต่ละตัว
- ฟังก์ชัน show ให้แสดงค่าทุกตัวรวมทั้ง base class ส่วน show ของ Topping ให้ทำเป็น pure virtual function
- operator string() โดยให้สามารถเรียกใช้ operator string() โดยใช้หลักการ dynamic binding ได้ด้วย แต่ละคลาสให้รีเทิร์นค่าดังนี้
 - o Topping ให้รีเทิร์น detail
 - o Chocolate ให้รีเทิร์น คำอธิบายชนิดของช็อกโกแลต เช่น ถ้า type=1 ให้รีเทิร์น "dark chocolate" เป็นต้น
 - o Fruit ให้รีเทิร์น name
- แต่ละคลาสจงสร้างคอนสตรัคเตอร์ให้สามารถแปลงชนิดข้อมูลจาก string ไปเป็นอ็อบเจกต์ของคลาสตนเองได้ (convert จาก string ไปเป็น object ของคลาส) โดยให้กำหนดค่าต่างๆ ดังนี้
 - o Topping กับ Chocolate ให้ detail มีค่าเท่ากับ string ที่รับเข้ามา ส่วนค่าอื่นให้กำหนดให้เป็นค่าเดียวกับ default constructor
 - o Fruit ให้ detail และ name มีค่าเท่ากับ string ที่รับเข้ามา ส่วนค่าอื่นให้กำหนดให้เป็นค่าเดียวกับ default constructor

คลาส Dessert, IceCream, Cake, IceCreamCake จงสร้าง

- default constructor ในการกำหนดค่าแอททริบิวต์เป็นอะไรก็ได้แต่ต้องสอดคล้องกับเงื่อนไขในฟังก์ชัน set โดยที่ topping ให้จองพื้นที่ตามค่า num โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน topping
- constructor กำหนดค่าแอททริบิวต์ทุกตัวรวมทั้ง base class เป็นค่าอะไรก็ได้แต่ต้องสอดคล้องกับเงื่อนไขในฟังก์ชัน set โดยที่ topping ให้จองพื้นที่ตามค่า num โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน topping
- destructor ในการคืนพื้นที่ และแสดงค่าในคลาสของตนเองออกทางจอภาพ
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอททริบิวต์แต่ละตัวโดยรับค่าจากพารามิเตอร์ โดยที่
 - o price, num, layer ต้องมากกว่าศูนย์
 - o ไอศกรีมมีแค่ 3 แบบ และเค้กมีแค่ 4 แบบเท่านั้น
 - o ถ้าค่า num เปลี่ยนจะต้องจองพื้นที่ใหม่ให้ topping ด้วย

- ให้กำหนดค่าให้สมาชิกที่อยู่ใน topping ทีละตัว โดยระบุตำแหน่งที่เก็บในอาร์เรย์ เช่น `setTopping(int, Topping*)` เป็นต้น
- ฟังก์ชัน `get` ในการรีเทิร์นค่าของแอทริบิวต์แต่ละตัว กรณี topping ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ทีละ 1 ค่าโดยระบุตำแหน่งที่เก็บในอาร์เรย์
- ฟังก์ชัน `show` ให้แสดงค่าทุกค่ารวมทั้ง base class ส่วน `show` ของ `Dessert` ให้ทำเป็น pure virtual function
- operator `++` ให้ทำเป็น prefix และเป็นสมาชิกของคลาส โดยให้สามารถใช้หลักการ dynamic binding ได้ด้วย โดยที่แต่ละคลาสให้เพิ่มค่าดังนี้
 - `Dessert` ให้เพิ่มค่า `price` ทีละ 10
 - `IceCream` ให้เพิ่มค่า `type` ถ้าเดิมเป็น 1 ให้ปรับเป็น 2 ถ้าเดิมเป็น 2 ให้ปรับเป็น 3 ถ้าเดิมเป็น 3 ให้ปรับเป็น 1
 - `Cake` ให้เพิ่มค่า `type` เช่นเดียวกับกรณี `IceCream` แต่มีทั้งหมด 4 ค่า
 - `IceCreamCake` ให้เพิ่ม `layer` ทีละ 1

คลาส `IceCream` จงสร้าง

- ฟังก์ชันในการนับจำนวนอ็อบเจกต์ของคลาส `IceCream` ว่า ณ ปัจจุบันมีจำนวนอ็อบเจกต์ของ `IceCream` อยู่ทั้งหมดเท่าไร ถึงแม้ว่าจะไม่มีอ็อบเจกต์ของ `IceCream` อยู่เลยก็ต้องสามารถบอกได้ด้วย

คลาส `Dessert` จงสร้าง

- ฟังก์ชัน `showTopping()` ในการแสดงค่าแอทริบิวต์ทุกค่าที่เก็บใน topping โดยใช้หลักการ dynamic binding

คลาส `IceCreamCake` จงสร้าง

- copy constructor ในการก๊อปปี้ค่าทุกค่าอย่างถูกต้อง
- function overloading ในการกำหนดค่าต่างๆ

คลาส `FoodBox`, `SquareFoodBox`, `RoundFoodBox` จงสร้าง

- default constructor
 - `FoodBox` จองพื้นที่ให้กับ `dessert` และ `foodDetail` ให้มีขนาด `num` เท่ากับ 1 โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน `dessert` และ `foodDetail`
 - `SquareFoodBox` ยาว 30 กว้าง 15 สูง 8 จองพื้นที่ให้กับ `dessert` และ `foodDetail` ให้มีขนาด `num` เท่ากับ 3 โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน `dessert` และ `foodDetail`
 - `RoundFoodBox` เส้นผ่าศูนย์กลาง 15 สูง 8 จองพื้นที่ให้กับ `dessert` และ `foodDetail` ให้มีขนาด `num` เท่ากับ 2 โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน `dessert` และ `foodDetail`
- constructor กำหนดค่าให้กับแอทริบิวต์ทุกตัวโดยการกำหนดจากพารามิเตอร์ที่รับมา โดยต้องมียกค่ามากกว่าศูนย์ และ จองพื้นที่ให้กับ `dessert` และ `foodDetail` ให้มีขนาดเท่ากับ `num` โดยที่ค่า `num` ต้องมากกว่า 0 โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน `dessert` และ `foodDetail`

- destructor ในการคืนพื้นที่ และแสดงค่าในคลาสของตนเอง
ออกทางจอภาพ
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอตทริบิวต์แต่ละตัวโดยรับค่าจากพารามิเตอร์
โดยที่
 - o แอตทริบิวต์ทุกตัวที่เป็น int ต้องมีค่ามากกว่าศูนย์
 - o ถ้าค่า num เปลี่ยนจะต้องจองพื้นที่ใหม่ให้ dessert และ foodDetail
ด้วย
 - o ให้กำหนดค่าให้สมาชิกที่อยู่ใน dessert ที่ละตัว โดยระบุตำแหน่งที่เก็บ
ในอาร์เรย์ เช่น setDesert(int, Desert*) เป็นต้น
 - o ให้กำหนดค่าให้สมาชิกที่อยู่ใน foodDetail ที่ละตัว โดยระบุตำแหน่งที่
เก็บในอาร์เรย์ เช่น setDetail(int, string) เป็นต้น
- ฟังก์ชัน get ในการรีเทิร์นค่าของแอตทริบิวต์แต่ละตัว กรณี dessert กับ
foodDetail ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ที่ละ 1 ค่าโดยระบุตำแหน่งที่เก็บใน
อาร์เรย์
- ฟังก์ชัน show ให้แสดงค่าทุกค่ารวมทั้ง base class ส่วน show ของ FoodBox
ให้ทำเป็น pure virtual function
- operator << ในการพิมพ์ค่าทุกค่าของคลาสตนเองและของ superclass ออก
ทางจอภาพ โดยใช้หลักการ dynamic binding ให้ทำเฉพาะคลาสเหล่านี้
เท่านั้น คลาสอื่นห้ามใช้ <<

คลาส FoodBox จงสร้าง

- operator > ในการเปรียบเทียบว่า ขนมในกล่องแรกแพงกว่ากล่องที่สอง
หรือไม่
- operator <= ในการเปรียบเทียบว่า ขนมในกล่องแรกถูกกว่าหรือเท่ากับกล่อง
ที่สองหรือไม่ ให้ทำโดยไปเรียกใช้ operator >

ใน main

- จงสร้างอ็อบเจกต์ดังนี้

ชื่ออ็อบเจกต์	คลาส	รายละเอียด
dark	Chocolate	type=1 price=30 detail="gradeA"
white	Chocolate	type=2 price=35 detail="grade premium"
milk	Chocolate	type=3 price=25 detail="from Chiang Mai"
cherry	Fruit	name="Cherry" price=20 detail="premium fruit"
strawberry	Fruit	name="Strawberry" price=30 detail="sweet"
icecream	IceCream	type=เจลาโต้ flavor="Tiramisu" price=90 ใส่ dark
butterCake	Cake	type=butter flavor="Coffee" price=120 ใส่ white และ strawberry
iceCake1	IceCreamCake	Layer=5 typeเค้ก=sponge typeไอติม=ไอศกรีม ธรรมดา flavor="vanilla ice cream and orange cake" price=240 ใส่ milk และ cherry
iceCake2	IceCreamCake	3 เลเยอร์ typeเค้ก=butter typeไอติม=ไอศกรีมเจลา โต้ flavor="Pistachio ice cream and vanilla cake" price=250 ใส่ strawberry

squareBox	SquareFoodBox	ยาว 30 กว้าง 15 สูง 8 ใส่ icecream, butterCake, iceCake1 foodDetail ให้กำหนดค่าตามลำดับดังนี้ "Tiramisu ice cream with dark chocolate" "Coffee cake with white chocolate and strawberry" "Ice cream cake + milk + cherry"
roundBox	RoundFoodBox	เส้นผ่าศูนย์กลาง 20 สูง 10 ใส่ iceCake2 foodDetail ให้กำหนดค่าตามลำดับดังนี้ "Ice cream cake + strawberry"

- จงแสดงค่าของ dessert ทั้งหมดที่อยู่ใน squareBox และ roundBox โดยใช้หลักการ dynamic binding
- จงแปลง dark, white, milk, cherry, strawberry ให้เป็น string โดยเก็บผลลัพธ์ที่เป็น string ไว้ที่ darkS, whiteS, milkS, cherryS, strawberryS ตามลำดับ จากนั้นให้แสดงผลลัพธ์ของการแปลงทั้งหมดออกทางจอภาพ
- จงสร้าง dark2S และ raspberryS เป็น string โดยกำหนดค่าให้เป็น "super premium" และ "raspberry" ตามลำดับ จากนั้นให้แปลงค่าไปเป็นอ็อบเจกต์ของคลาส Chocolate และ Fruit โดยตั้งชื่อว่า dark2 และ raspberry ตามลำดับ จากนั้น ให้แสดงค่าทั้งหมดของอ็อบเจกต์ทั้งสองออกทางจอภาพ
- จงเรียกใช้ copy constructor ในการก๊อปปี้ค่า iceCake2 ไปเก็บไว้ในอ็อบเจกต์ตัวใหม่ที่ชื่อ iceCake3 จากนั้นให้เปลี่ยนชนิดของเค้กของอ็อบเจกต์ iceCake3 เป็น sponge และแสดงผลลัพธ์ของอ็อบเจกต์ที่ก๊อปปี้มานี้ออกทางจอภาพ
- จงเปลี่ยนที่ออปปีงของ iceCake3 ให้เป็น dark2 และ raspberry จากนั้นให้แสดงค่าทุกค่าของ iceCake3 ออกทางจอภาพ
- จงสร้าง dessertArray ให้เป็นอาร์เรย์ขนาด 5 เก็บพอยเตอร์ที่ชี้ไปยัง Dessert โดยให้สมาชิกแต่ละตัวในอาร์เรย์ชี้ไปยัง icecream, butterCake, iceCake1, iceCake2, iceCake3 ตามลำดับ จากนั้นให้เรียกใช้ operator++ โดยใช้หลักการ dynamic binding จากนั้น ให้แสดงค่าของสมาชิกทุกตัวใน dessertArray โดยใช้หลักการ dynamic binding ด้วย
- จงแสดงค่าทั้งหมดของ squareBox และ roundBox โดยการเรียกใช้ operator<<
- จงตรวจสอบว่าขนมทั้งหมดที่อยู่ใน squareBox ราคาแพงกว่า ขนมทั้งหมดที่อยู่ใน roundBox หรือไม่ ให้เปรียบเทียบทั้งสองอ็อบเจกต์โดยใช้ operator<= และให้พิมพ์ออกมาด้วยว่ากล่องไหนแพงกว่ากัน
- ระหว่าง squareBox กับ roundBox กล่องใดมีจำนวนขนมมากกว่ากัน

ឧទាហរណ៍

```
#include<iostream>
using namespace std;
class Topping {
    string detail;
    double price;
public:
    virtual void show() = 0;
    virtual ~Topping() {}
    Topping() {
        detail = "no detail";
        price = 0;
    }
    Topping(string s, double p) {
        detail = s;
        if (p>0)
            price = p;
        else
            price = 1;
    }
    Topping(string s) {
        detail = s;
        price=0;
    }
    string getDetail() { return detail; }
    double getPrice() { return price; }
    virtual operator string() { return detail; }
};

class Chocolate: public Topping {
    int type; // 1.dark 2.white 3.milk
public:
    Chocolate(): Topping("made in Swiss", 40) { type = 1; }
    Chocolate(int t, string s, double p): Topping(s, p) {
        if (t>0 && t<4)
            type = t;
        else
            type = 1;
    }
    Chocolate(string s): Topping(s, 40) { type=1; }
    int getType() { return type; }
    string getTypeString() {
        switch (type) {
            case 1: return "dark chocolate";
```

```

        case 2: return "white chocolate";
        case 3: return "milk chocolate";
        default: return "dark chocolate";
    }
}
void show() {
    cout<<"Type: "<<getTypeString()<<endl;
    cout<<"Detail: "<<getDetail()<<endl;
    cout<<"Price: "<<getPrice()<<endl;
}
operator string() { return getTypeString(); }
};

class Fruit: public Topping {
    string name;
public:
    Fruit(): Topping("sweet", 30) { name = "Strawberry"; }
    Fruit(string n, string s, double p): Topping(s, p) { name = n; }
    Fruit(string s): Topping(s, 30) { name = s; }
    string getName() { return name; }
    void show() {
        cout<<"Name: "<<name<<endl;
        cout<<"Price: "<<getPrice()<<endl;
        cout<<"Detail: "<<getDetail()<<endl;
    }
    operator string() { return name; }
};

class Dessert {
    int price;
    string flavor;
    Topping **topping;
    int num;
public:
    virtual void show() = 0;
    Dessert() {
        setPrice(1);
        setFlavor("delicious");
        num=1;
        topping = new Topping*[num];
    }
    Dessert(int p, string f, int n) {
        setPrice(p);
        setFlavor(f);
    }
};

```

```

        if (n<=0)
            n=1;
        num = n;
        topping = new Topping*[num];
    }
    virtual ~Dessert() {
        cout<<"\nDESTRUCTOR CLASS DESSERT"<<endl;
        showDessert();
        delete[] topping;
    }
    void setNum(int i) {
        if (i<=0)
            i=1;
        if (i==num)
            return;
        num = i;
        delete[] topping;
        topping = new Topping*[num];
    }
    void setPrice(double p) {
        if (p>0)
            price=p;
        else
            price=1;
    }
    void setFlavor(string f) { flavor = f; }
    void setTopping(int i, Topping *t) { topping[i] = t; }
    double getPrice() { return price; }
    string getFlavor() { return flavor; }
    int getNum() { return num; }
    Topping* getTopping(int i) { return topping[i]; }
    void showTopping() {
        for (int i=0; i<num; i++)
            topping[i]->show();
    }
    void showDessert() {
        cout<<"Price = "<<price<<endl;
        cout<<"Flavor = "<<flavor<<endl;
        cout<<"amount of topping = "<<num<<endl;
        showTopping();
    }
    virtual void operator++() { setPrice(price+10); }
};

```

```

class IceCream: virtual public Dessert {
    int type; //1.normal 2.gelato 3.yogurt
    static int count;
public:
    IceCream() {
        setType(1);
        count++;
    }
    IceCream(int t, double p, string f, int n): Dessert(p,f,n) {
        setType(t);
        count++;
    }
    ~IceCream() {
        cout<<"\nDESTRUCTOR CLASS ICECREAM"<<endl;
        showIceCream();
        count--;
    }
    void setType(int i) {
        if (i>0 && i<4)
            type=i;
        else
            type=1;
    }
    int getType() { return type; }
    void show() {
        cout<<"\nICE CREAM"<<endl;
        showIceCream();
        showDessert();
    }
    void showIceCream() {
        cout<<"Ice Cream type (1.normal 2.gelato 3.yogurt): "<<type<<endl;
    }
    void operator++() {
        type++;
        if (type==4)
            type=1;
    }
    static int getCount() { return count; }
};

int IceCream::count;

```

```

class Cake: virtual public Dessert {
    int type; //1.butter 2.foam 3.mousse 4.sponge
public:
    Cake() { setType(1); }
    Cake(int t, double p, string f, int n): Dessert(p,f,n) { setType(t); }
    ~Cake() {
        cout<<"\nDESTRUCTOR CLASS CAKE"<<endl;
        showCake();
    }
    void setType(int i) {
        if (i>0 && i<5)
            type=i;
        else
            type=1;
    }
    int getType() { return type; }
    void show() {
        cout<<"\nCAKE"<<endl;
        showCake();
        showDessert();
    }
    void showCake() {
        cout<<"Cake type (1.butter 2.foam 3.mousse 4.sponge): "<<type<<endl;
    }
    void operator++() {
        type++;
        if (type==5)
            type=1;
    }
};

```

```

class IceCreamCake: public IceCream, public Cake {
    int layer;
public:
    IceCreamCake() { setLayer(3); }
    IceCreamCake(int l, int cake, int ice, double p, string f, int n): Dessert(p,f,n) {
        Cake::setType(cake);
        IceCream::setType(ice);
        setLayer(l);
    }
    IceCreamCake(IceCreamCake& c): Dessert(c.getPrice(), c.getFlavor(), c.getNum()) {
        Cake::setType(c.Cake::getType());
        IceCream::setType(c.IceCream::getType());
        setLayer(c.getLayer());
    }
};

```

```

        for (int i=0; i<getNum(); i++) {
            setTopping(i, c.getTopping(i));
        }
    }
    ~IceCreamCake() {
        cout<<"\nDESTRUCTOR CLASS ICECREAMCAKE"<<endl;
        cout<<"Layer: "<<layer<<endl;
    }
    void setLayer(int i) {
        if (i>0)
            layer=i;
        else
            layer=3;
    }
    void setLayer(int i, double p) {
        setLayer(i);
        setPrice(p);
    }
    int getLayer() { return layer; }
    void show() {
        cout<<"\nICE CREAM CAKE"<<endl;
        showIceCreamCake();
        showIceCream();
        showCake();
        showDessert();
    }
    void showIceCreamCake() { cout<<"Layer: "<<layer<<endl; }
    void operator++() { layer++; }
};

```

```

class FoodBox {
    Dessert **dessert;
    int num;
    string *foodDetail;
public:
    virtual void show() = 0;
    FoodBox() {
        num=1;
        dessert = new Dessert*[num];
        foodDetail = new string[num];
    }
}

```

```

FoodBox(int n) {
    if (n<=0)
        n=1;
    num = n;
    dessert = new Dessert*[num];
    foodDetail = new string[num];
}
virtual ~FoodBox() {
    cout<<"\nDESTRUCTOR CLASS FOODBOX"<<endl;
    showFoodBox();
    delete[] dessert;
    delete[] foodDetail;
}
void setNum(int i) {
    if (i<=0)
        i=1;
    if (i==num)
        return;
    num = i;
    delete[] dessert;
    delete[] foodDetail;
    dessert = new Dessert*[num];
    foodDetail = new string[num];
}
void setDessert(int i, Dessert *d) { dessert[i]=d; }
void setfoodDetail(int i, string s) { foodDetail[i]=s; }
int getNum() { return num; }
Dessert* getDessert(int i) { return dessert[i]; }
string getFoodDetail(int i) { return foodDetail[i]; }
void showFoodBox() {
    for (int i=0; i<num; i++) {
        cout<<"\nFood detail: "<<foodDetail[i]<<endl;
        dessert[i]->show();
    }
}
double getTotalPrice() {
    double total=0;
    for (int i=0; i<num; i++) {
        total+=dessert[i]->getPrice();
        for (int j=0; j<dessert[i]->getNum(); j++)
            total+=dessert[i]->getTopping(j)->getPrice();
    }
    return total;
}

```

```

    bool operator > (FoodBox &b) {
        if (getTotalPrice() > b.getTotalPrice())
            return true;
        else
            return false;
    }
    bool operator <= (FoodBox &b) {
        if ((*this) > b)
            return false;
        else
            return true;
    }
};

class SquareFoodBox: public FoodBox {
    int width;
    int height;
    int length;
public:
    SquareFoodBox(): FoodBox(3) {
        setLength(30);
        setWidth(15);
        setHeight(8);
    }
    SquareFoodBox(int w, int h, int l, int n): FoodBox(n) {
        setLength(w);
        setWidth(h);
        setHeight(n);
    }
    ~SquareFoodBox() {
        cout << "\nDESTRUCTOR CLASS SQUAREFOODBOX" << endl;
        showSquare();
    }
    void setWidth(int w) {
        if (w > 0)
            width = w;
        else
            width = 1;
    }
    void setHeight(int h) {
        if (h > 0)
            height = h;
        else
            height = 1;
    }
};

```

```

    }
    void setLength(int l) {
        if (l>0)
            length=l;
        else
            length=1;
    }
    int getWidth() { return width; }
    int getHeight() { return height; }
    int getLength() { return length; }
    void showSquare() {
        cout<<"\nSQUARE BOX"<<endl;
        cout<<"width="<<width<<endl;
        cout<<"height="<<height<<endl;
        cout<<"length="<<length<<endl;
    }
    void show() {
        showSquare();
        showFoodBox();
    }
};

```

```

class RoundFoodBox: public FoodBox {
    int diameter;
    int height;
public:
    RoundFoodBox(): FoodBox(2) {
        setDiameter(15);
        setHeight(8);
    }
    RoundFoodBox(int d, int h, int n): FoodBox(n) {
        setDiameter(d);
        setHeight(h);
    }
    ~RoundFoodBox() {
        cout<<"\nDESTRUCTOR CLASS ROUNDFOODBOX"<<endl;
        showRound();
    }
    void setDiameter(int d) {
        if (d>0)
            diameter=d;
        else
            diameter=1;
    }
}

```

```

void setHeight(int h) {
    if (h>0)
        height=h;
    else
        height =1;
}
int getDiameter() { return diameter; }
int getHeight() { return height; }
void showRound() {
    cout<<"\nROUND BOX"<<endl;
    cout<<"diameter="<<diameter<<endl;
    cout<<"height="<<height<<endl;
}
void show() {
    showRound();
    showFoodBox();
}
};

ostream& operator<<(ostream &o, FoodBox &f) {
    f.show();
    return o;
}

int main() {
    Chocolate dark(1,"gradeA",30);
    Chocolate white(2,"grade premium",35);
    Chocolate milk(3,"from Chiang Mai",25);
    Fruit cherry("Cherry","premium fruit",20);
    Fruit strawberry;

    IceCream icecream(2,90,"Tiramisu",1);
    icecream.setTopping(0, &dark);

    Cake butterCake(1,120,"Coffee",2);
    butterCake.setTopping(0, &white);
    butterCake.setTopping(1, &strawberry);

    IceCreamCake iceCake1(5,4,1,240,"vanilla ice cream and orange cake",2);
    iceCake1.setTopping(0, &milk);
    iceCake1.setTopping(1, &cherry);

    IceCreamCake iceCake2(3,1,2,250,"Pistachio ice cream and vanilla cake",1);
    iceCake2.setTopping(0, &strawberry);

    SquareFoodBox squareBox;

```

```

squareBox.setDessert(0, &icecream);
squareBox.setDessert(1, &butterCake);
squareBox.setDessert(2, &iceCake1);
squareBox.setfoodDetail(0, "Tiramisu ice cream with dark chocolate");
squareBox.setfoodDetail(1, "Coffee cake with white chocolate and strawberry");
squareBox.setfoodDetail(2, "Ice cream cake + milk + cherry");

```

```

RoundFoodBox roundBox(20, 10, 1);
roundBox.setDessert(0, &iceCake2);
roundBox.setfoodDetail(0, "Ice cream cake + strawberry");

```

```

cout<<"All dessert in squareBox....."<<endl;
for (int i=0; i<squareBox.getNum(); i++)
    squareBox.getDessert(i)->show();
cout<<"\nAll dessert in roundBox....."<<endl;
for (int i=0; i<roundBox.getNum(); i++)
    roundBox.getDessert(i)->show();

```

```

cout<<"\nConvert all topping to string....."<<endl;
string darkS = dark;
string whiteS = white;
string milkS = milk;
string cherryS = cherry;
string strawberryS = strawberry;
cout<<darkS<<endl;
cout<<whiteS<<endl;
cout<<milkS<<endl;
cout<<cherryS<<endl;
cout<<strawberryS<<endl;

```

```

cout<<"\nConvert from string to object....."<<endl;
string dark2S, raspberryS;
dark2S="super premium";
raspberryS="raspberry";
Chocolate dark2 = dark2S;
Fruit raspberry = raspberryS;
dark2.show();
raspberry.show();

```

```

cout<<"\nCall copy constructor iceCake2 to iceCake3....."<<endl;
IceCreamCake iceCake3(iceCake2);
iceCake3.Cake::setType(4);
iceCake3.show();

```

```

cout<<"\nUpdate topping dark2 and raspberry to iceCake3....."<<endl;
iceCake3.setNum(2);
iceCake3.setTopping(0, &dark2);
iceCake3.setTopping(1, &raspberry);
iceCake3.show();

cout<<"\nCreate dessertArray and call operator++....."<<endl;
Dessert *dessertArray[5] = {&icecream, &butterCake, &iceCake1, &iceCake2, &iceCake3};
for (int i=0; i<5; i++)
    ++(*dessertArray[i]);
for (int i=0; i<5; i++)
    dessertArray[i]->show();

cout<<"\ncall operator<< to show squareBox and roundBox....."<<endl;
cout<<squareBox<<endl;
cout<<roundBox<<endl;

cout<<"\nCompare price....."<<endl;
if (squareBox <= roundBox)
    cout<<"squareBox <= roundBox"<<endl;
else
    cout<<"squareBox > roundBox"<<endl;

cout<<"\nCompare number of dessert....."<<endl;
if (squareBox.getNum()>roundBox.getNum())
    cout<<"squareBox > roundBox"<<endl;
else
    cout<<"squareBox <= roundBox"<<endl;

cout<<"\n-----End-----"<<endl;
}

```

